

Product Requirements Document

AlignOS: Slack-Native Live Memory, Decision Intelligence, and Conflict Detection Agent

1. Product Summary

AlignOS is a Slack-native AI agent that turns messy team conversations into verified organizational memory. It detects decisions, extracts tasks, identifies conflicts, answers questions using real-time Slack evidence, and maintains a live project memory layer.

Unlike a normal Slack chatbot, AlignOS does not simply wait for a user to ask questions. It continuously observes project conversations, detects meaningful events, asks for confirmation when needed, updates a structured memory database, and alerts teams when new messages contradict previous decisions.

The product combines:

- Slack Agent/App experience for the user interface
- Real-Time Search API for live Slack context retrieval
- RAG-based evidence verification for grounded answers
- MCP server tools for modular decision, memory, and conflict operations
- A live memory database for confirmed decisions, tasks, blockers, and project state

2. One-Line Pitch

AlignOS is a Slack-native intelligence layer that converts team conversations into verified decisions, live memory, and conflict-free project alignment.

3. Problem Statement

Teams use Slack for daily work, but important information gets buried inside long conversations, threads, files, and channels.

Common problems:

1. Decisions are made casually and then forgotten.
2. New team members cannot understand why something was chosen.
3. People contradict old decisions without realizing it.
4. Tasks and blockers are scattered across messages.
5. AI chatbots often hallucinate because they answer without checking live workspace evidence.
6. Teams waste time asking, "What did we decide?", "Who owns this?", "Is this final?", and "Where was this discussed?"

Slack already contains the truth, but that truth is unstructured.

AlignOS solves this by converting Slack conversation into structured, evidence-backed team memory.

4. Target Users

4.1 Primary Users

Hackathon teams

Small groups building projects quickly. They need decision tracking, task extraction, and quick summaries.

Startup/product teams

Teams making rapid decisions in Slack and needing lightweight project memory without heavy project-management tools.

Engineering teams

Teams discussing technical decisions, bugs, deployments, and architecture in Slack.

Student project groups

Students collaborating on assignments, presentations, research, or software projects.

4.2 Secondary Users

Managers/team leads

Need visibility into decisions, blockers, ownership, and project direction.

New joiners

Need to understand project history without reading hundreds of messages.

Documentation owners

Need help converting Slack conversations into clean knowledge bases.

5. Product Goals

Goal 1: Capture decisions automatically

Detect when a team appears to make a decision and ask users to confirm it.

Goal 2: Build live project memory

Store confirmed decisions, tasks, blockers, deadlines, and unresolved questions in a structured database.

Goal 3: Provide evidence-backed answers

When users ask questions, retrieve relevant Slack context and memory before answering.

Goal 4: Detect conflicts

Alert the team when a new message seems to contradict confirmed project memory.

Goal 5: Reduce hallucination

Only answer confidently when the answer is supported by Slack evidence or confirmed memory.

Goal 6: Stay inside Slack

Users should not need to open another dashboard for the MVP. The product should work through Slack messages, buttons, modals, and cards.

6. Non-Goals

For the MVP, AlignOS will not:

1. Replace Jira, Linear, Notion, or Confluence completely.
2. Build a full web dashboard.
3. Analyze every private channel unless permission is granted.
4. Automatically make irreversible decisions without human confirmation.
5. Store complete Slack message history externally.
6. Guarantee perfect legal/compliance-grade audit logs.
7. Support every enterprise integration from day one.

7. Core Product Concept

AlignOS has two types of memory:

7.1 Live Slack Context

This comes from Slack Real-Time Search API.

It is used for:

- latest messages
- recent decisions
- thread context
- files
- relevant channel history
- current team discussion

This is the raw, fresh evidence layer.

7.2 Verified Memory Database

This is AlignOS's own structured memory.

It stores:

- confirmed decisions
- decision reasons
- owners
- tasks
- blockers
- deadlines
- conflicts
- evidence message references
- project status summaries

This is the clean, long-term memory layer.

The core intelligence comes from combining both.

8. Example User Story

A team is discussing a hackathon project in Slack.

Messages:

Ayush: Should we use PostgreSQL or MongoDB?

Rahul: MongoDB may be faster for prototyping.

Priya: PostgreSQL is safer because our data is structured.

Ayush: Okay, final then. PostgreSQL for v1.

Rahul: Fine, I'll use PostgreSQL.

AlignOS detects a possible decision:

Possible decision detected:

Decision: Use PostgreSQL for v1

Reason: Structured project data

Confidence: High

Evidence: 5 related messages

Buttons:

[Confirm] [Edit] [Reject]

After confirmation, AlignOS stores this in memory.

Later Rahul says:

I'll start MongoDB setup.

AlignOS detects a conflict:

Possible conflict detected:

Your message mentions MongoDB, but confirmed project memory says: Database = PostgreSQL for v1.

Actions:

[Remind Decision] [Reopen Decision] [Ignore]

Later a user asks:

@AlignOS why did we choose PostgreSQL?

AlignOS searches Slack context and memory, then answers:

The team chose PostgreSQL for v1 because the data model is structured. This decision was confirmed after Priya suggested PostgreSQL and Ayush finalized it. Rahul agreed afterward.

9. Key Features

9.1 Feature 1: Evidence-Backed Question Answering

Description

Users can ask AlignOS questions about project history, decisions, tasks, or context.

Example:

@AlignOS what did we decide about the database?

AlignOS retrieves relevant Slack evidence using Real-Time Search API, checks confirmed memory, and generates a grounded answer.

User Flow

1. User mentions AlignOS in Slack.
2. Backend receives app mention event.
3. Intent router classifies the message as a project-memory question.
4. Backend searches confirmed memory.
5. Backend calls Real-Time Search API for live Slack evidence.
6. RAG evidence checker validates support.
7. LLM generates answer.
8. Bot replies in Slack.

Acceptance Criteria

- The agent can answer questions using retrieved Slack messages.
- The answer includes confidence level: High, Medium, or Low.

- If evidence is insufficient, the agent says it cannot confirm.
- The answer should not invent unsupported facts.
- The answer should mention whether the result came from confirmed memory or live discussion.

Example Response

Confirmed decision: PostgreSQL for v1.

Reason: The team agreed that PostgreSQL fits the structured task data better.

Confidence: High

Source: Confirmed memory + recent Slack discussion

9.2 Feature 2: Automatic Decision Detection

Description

AlignOS detects possible decisions from Slack messages and threads.

Decision-like phrases include:

- "Let's finalize..."
- "We agreed on..."
- "Final decision..."
- "Okay, we'll use..."
- "Let's go with..."
- "Deadline is fixed for..."
- "We are dropping..."

User Flow

1. A user sends a message in a monitored channel.
2. Slack sends message event to backend.
3. Backend retrieves nearby thread/channel context.
4. MCP tool `detect_decision` analyzes the message and context.
5. If a decision is detected, AlignOS posts a confirmation card.
6. User confirms, edits, or rejects.
7. Confirmed decisions are saved to memory.

Acceptance Criteria

- The system detects clear decision statements.
- The system does not save decisions without confirmation.
- The confirmation card contains decision summary, reason, participants, and confidence.
- The user can confirm or reject the decision.
- Confirmed decisions are stored with evidence references.

Slack Card Example

Possible decision detected:

Decision: Use PostgreSQL for v1.

Reason: Better fit for structured project data.

Confidence: High

Actions: [Confirm] [Edit] [Reject]

9.3 Feature 3: Live Memory Database

Description

Confirmed decisions and project information are stored in a structured database.

Memory types:

- decisions
- tasks
- blockers
- deadlines
- unresolved questions
- conflicts
- project summaries

User Flow

1. Decision is confirmed.
2. Backend calls MCP tool `save_decision`.
3. Database stores the decision.
4. AlignOS posts confirmation in Slack.
5. Future answers and conflict checks use this memory.

Acceptance Criteria

- Confirmed decisions are retrievable later.
- Each memory item has evidence references.
- Each memory item has a status: proposed, confirmed, rejected, reopened, superseded.
- Memory can be queried by topic.
- Memory should be workspace/channel scoped.

9.4 Feature 4: Conflict Detection

Description

AlignOS detects when a new message contradicts confirmed memory.

Example:

Confirmed memory: Database = PostgreSQL

New message: I'll start MongoDB setup.

AlignOS flags this as a possible conflict.

User Flow

1. User sends a message.
2. Backend checks if the message mentions topics in memory.
3. Backend retrieves relevant memory items.
4. Backend optionally calls Real-Time Search API to check if a newer decision changed the context.
5. MCP tool `detect_conflict` compares new message with memory and recent context.
6. If conflict is found, bot posts an alert.

Acceptance Criteria

- The system detects obvious contradictions.
- The system avoids false alarms for casual brainstorming.
- The alert includes the confirmed memory item being contradicted.
- Users can choose: Remind Decision, Reopen Decision, or Ignore.
- Ignored conflicts should be logged to reduce repeated alerts.

Slack Alert Example

Possible conflict detected.

This message mentions MongoDB, but confirmed project memory says: Database = PostgreSQL for v1.

Actions: [Remind Decision] [Reopen Decision] [Ignore]

9.5 Feature 5: Project Memory Summary

Description

Users can ask AlignOS for a project state summary.

Example:

@AlignOS show project memory

The bot returns:

- current goal
- confirmed decisions
- open tasks
- blockers
- unresolved questions
- recent conflicts
- upcoming deadlines

Acceptance Criteria

- Summary should be clean and skimmable.
- Summary should separate confirmed facts from uncertain items.
- Summary should include only relevant project/channel memory.
- Summary should update when new decisions are confirmed.

Example Output

Project Memory: Hackathon Demo

Current Goal: Build AlignOS MVP for Slack hackathon.

Confirmed Decisions: 1. Use PostgreSQL for memory database. 2. Build Slack-first MVP, no separate dashboard. 3. Use Real-Time Search API for live evidence.

Open Tasks: 1. Set up Slack event endpoint. 2. Build MCP decision tools. 3. Add conflict detection.

Blockers: 1. Need Real-Time Search API access. 2. Need Slack app permissions confirmed.

Unresolved Questions: 1. Should we add GitHub integration? 2. Should memory export to Notion be included in demo?

9.6 Feature 6: No-Evidence Refusal

Description

If the user asks something that is not supported by memory or Slack evidence, AlignOS should not hallucinate.

Example:

User: @AlignOS did we finalize pricing?

If no evidence exists:

I could not find enough evidence that pricing was finalized. I found discussion about possible pricing options, but no confirmed decision.

Would you like me to start a decision thread?

Acceptance Criteria

- The system refuses unsupported claims.
- The system distinguishes between “discussed” and “confirmed.”
- The system suggests a next action when evidence is missing.
- The system should not sound overly confident when evidence is weak.

9.7 Feature 7: Decision Reopening

Description

Teams should be able to reopen or supersede old decisions.

Example:

@AlignOS reopen database decision

User Flow:

1. User asks to reopen a decision.
2. AlignOS finds matching confirmed decision.
3. Bot posts decision details.
4. User confirms reopening.
5. Decision status changes to reopened.
6. New decision can later supersede the old one.

Acceptance Criteria

- Old decisions are not deleted.
- Superseded decisions remain visible in history.
- New decisions can reference old decisions.
- Conflict detector respects latest confirmed decision.

10. MVP Scope

The MVP should include:

1. Slack app mention support
2. Slack message event listener
3. Real-Time Search API retrieval
4. MCP client connected to custom MCP server
5. Tools:
 6. detect_decision
 7. save_decision
 8. search_memory
 9. detect_conflict
 10. verify_evidence
 11. PostgreSQL/Supabase memory database
 12. Confirmation buttons
 13. Conflict alert buttons
 14. Project memory summary command
 15. Evidence-backed Q&A

11. Post-MVP Features

After MVP, add:

1. GitHub integration through MCP
2. Jira/Linear ticket creation
3. Notion/Google Docs decision export
4. Slack Canvas generation
5. Weekly decision digest
6. Multi-channel project graph
7. Web dashboard
8. Team analytics
9. Decision quality score
10. Automatic onboarding summary for new members

12. User Personas

Persona 1: Ayush, Hackathon Builder

Ayush is building a hackathon project with teammates in Slack. The team discusses decisions quickly and often forgets what was finalized.

Needs:

- Quick decision capture
- Task tracking
- Conflict detection
- Demo-friendly Slack workflow

Pain:

- Too much scrolling
- Repeated confusion
- No time to manually document everything

Persona 2: Priya, Product Manager

Priya manages fast-moving projects.

Needs:

- See project decisions
- Know blockers
- Understand why choices were made
- Keep team aligned

Pain:

- Decisions buried in threads
- People act on outdated information
- Documentation is always behind

Persona 3: Rahul, Engineer

Rahul works from Slack discussions and needs technical clarity.

Needs:

- Know final technical decisions
- Understand current architecture
- Avoid building the wrong thing
- Ask questions without reading old threads

Pain:

- Conflicting messages
- Unclear ownership
- Missing context

13. Functional Requirements

13.1 Slack Event Handling

The system must receive and process:

- app mentions
- channel messages
- thread replies
- button interactions
- modal submissions
- slash commands, optional

Requirements

- Verify Slack request signatures.
- Respond quickly to Slack event challenge requests.
- Avoid duplicate processing using event IDs.
- Ignore bot messages to prevent loops.
- Support workspace/channel scoping.

13.2 Intent Routing

The system must classify incoming messages into intents:

- question_answering
- possible_decision
- possible_conflict
- show_memory
- save_task
- summarize_thread
- unknown

Requirements

- Use rule-based detection for simple cases.
- Use LLM classification for ambiguous cases.
- Return structured JSON.
- Include confidence score.

13.3 Real-Time Search Retrieval

The system must query Slack context when needed.

Use cases:

- answering user questions
- verifying decisions
- checking conflicts
- summarizing recent project changes

Requirements

- Generate search query from user message.
- Include channel context where possible.
- Retrieve relevant messages/files/users/channels.
- Pass retrieved evidence to LLM.
- Do not answer beyond evidence.

13.4 MCP Tool Layer

The backend must call MCP tools for modular reasoning and memory operations.

Required MCP tools:

detect_decision

Input:

- message
- thread_context
- recent_channel_context

Output:

- is_decision
- title
- summary
- reason
- participants
- confidence
- evidence_ids

save_decision

Input:

- decision object
- workspace_id
- channel_id
- confirmed_by

Output:

- decision_id
- status

search_memory

Input:

- query
- workspace_id
- channel_id

Output:

- matching memory items

detect_conflict

Input:

- new_message
- relevant_memory
- recent_context

Output:

- is_conflict
- conflict_type
- explanation
- conflicting_memory_id
- severity

verify_evidence

Input:

- proposed_answer
- evidence_messages
- memory_items

Output:

- support_level
- missing_evidence
- contradictions
- final_confidence

13.5 Memory Database

The system must store confirmed memory.

Tables:

1. workspaces
2. channels
3. users
4. decisions
5. tasks
6. blockers
7. conflicts
8. evidence_links
9. memory_items
10. audit_events

13.6 Slack Interaction Cards

The system must post interactive cards for:

- decision confirmation
- conflict detection
- memory summary
- reopen decision
- insufficient evidence actions

Buttons:

Decision card:

- Confirm
- Edit
- Reject

Conflict card:

- Remind Decision
- Reopen Decision
- Ignore

Insufficient evidence card:

- Start Decision Thread

- Search Again
- Ignore

14. Non-Functional Requirements

14.1 Performance

- Slack event acknowledgment should happen within Slack's expected event-response window.
- Long AI processing should be handled asynchronously.
- Simple memory queries should complete quickly.
- Real-Time Search calls should be minimized and cached where appropriate.

14.2 Reliability

- Events should be idempotent.
- Duplicate Slack events should not create duplicate memory items.
- Failed LLM calls should return a graceful fallback.
- Failed MCP calls should not crash the Slack app.
- Database failures should be logged clearly.

14.3 Security

- Verify Slack signing secret for all requests.
- Store tokens securely in environment variables.
- Use least-privilege Slack scopes.
- Do not expose raw internal logs to users.
- Do not store full Slack history unless necessary.
- Store only memory objects and evidence references for MVP.
- Respect user/channel permissions.

14.4 Privacy

- Do not summarize private channels unless bot has permission.
- Do not expose evidence from channels the requesting user cannot access.
- Avoid storing sensitive content unnecessarily.
- Let workspace admins disable monitoring for specific channels.
- Provide a delete-memory option for admins.

14.5 Explainability

Every important answer should show:

- confidence level
- whether it came from confirmed memory or live search
- if evidence was insufficient
- whether conflicts exist

15. Suggested Tech Stack

Frontend/User Interface

- Slack app
- Slack Block Kit cards
- Slack app mentions
- Slack buttons/modals

Backend

Preferred:

- Node.js
- Slack Bolt SDK
- Express/Fastify

Alternative:

- Python FastAPI
- slack_sdk

AI Layer

- OpenAI / Anthropic / other LLM provider
- Structured JSON outputs
- RAG evidence prompts
- Guardrail prompts for no-evidence refusal

MCP Layer

- Custom MCP server
- MCP client in backend
- Tools for memory, decision, conflict, and verification

Database

- Supabase PostgreSQL
- Or plain PostgreSQL
- SQLite only for local demo

Deployment

- Render
- Railway
- Fly.io
- Vercel for simple backend routes if compatible
- ngrok for local Slack testing

16. High-Level Architecture

Slack Workspace ↓ Slack App / Agent Interface ↓ Backend Orchestrator ↓ Intent Router ↓ Real-Time Search API + Memory DB ↓ MCP Client ↓ Custom MCP Server Tools ↓ LLM Reasoning Layer ↓ Slack Response / Card / Alert

17. Detailed System Flow

17.1 Ask Question Flow

User: @AlignOS why did we choose PostgreSQL?

Flow:

1. Slack sends app mention event to backend.
2. Backend verifies Slack signature.
3. Backend acknowledges event.
4. Intent router classifies as memory question.
5. Backend queries memory DB for PostgreSQL/database decisions.
6. Backend calls Real-Time Search API for live Slack evidence.
7. Backend calls MCP tool `verify_evidence`.
8. LLM generates answer using only verified context.
9. Backend posts answer to Slack thread.

17.2 Decision Detection Flow

User: Okay final, PostgreSQL for v1.

Flow:

1. Slack sends message event.
2. Backend ignores if message is from a bot.
3. Backend retrieves nearby context.
4. Backend calls MCP tool `detect_decision`.
5. If confidence is high enough, backend posts decision confirmation card.
6. User clicks Confirm.
7. Backend calls MCP tool `save_decision`.
8. Decision is stored in database.
9. Bot posts "Decision saved."

17.3 Conflict Detection Flow

User: I'll start MongoDB setup.

Flow:

1. Slack sends message event.
2. Backend extracts possible topic: database.
3. Backend searches memory for database decisions.

4. Memory returns PostgreSQL decision.
5. Backend calls Real-Time Search API to check if newer messages changed the decision.
6. MCP tool `detect_conflict` compares new message, memory, and latest context.
7. Conflict is detected.
8. Bot posts alert card.
9. User chooses action.

18. Data Model

18.1 decisions

Fields:

- id
- workspace_id
- channel_id
- thread_ts
- title
- summary
- reason
- status
- confidence
- confirmed_by_user_id
- created_at
- updated_at
- supersedes_decision_id
- evidence_count

Statuses:

- proposed
- confirmed
- rejected
- reopened
- superseded

18.2 evidence_links

Fields:

- id
- memory_item_id
- source_type
- slack_channel_id
- slack_message_ts
- slack_thread_ts
- slack_user_id
- snippet
- created_at

18.3 conflicts

Fields:

- id
- workspace_id
- channel_id
- message_ts
- conflict_type
- severity
- new_message_summary
- conflicting_memory_id
- explanation
- status
- created_at

Statuses:

- open
- ignored
- resolved
- reopened_decision

18.4 tasks

Fields:

- id
- workspace_id
- channel_id
- title
- owner_user_id
- status
- due_date
- evidence_message_ts
- created_at
- updated_at

18.5 memory_items

Fields:

- id
- workspace_id
- channel_id
- type
- title
- summary
- status
- confidence

- created_at
- updated_at

Types:

- decision
- task
- blocker
- deadline
- question
- summary

19. LLM Behavior Requirements

The LLM must:

1. Return structured JSON for classification.
2. Separate confirmed facts from guesses.
3. Refuse to answer if evidence is insufficient.
4. Mention uncertainty clearly.
5. Avoid inventing unsupported decisions.
6. Use Slack evidence and memory as primary context.
7. Prefer latest confirmed memory over older raw messages.
8. Detect contradiction between new messages and confirmed memory.
9. Avoid exposing private evidence from unauthorized contexts.

20. Prompting Strategy

20.1 Decision Detection Prompt

Input:

- current message
- thread context
- recent messages

Output JSON:

```
{ "is_decision": true, "title": "Use PostgreSQL for v1", "summary": "The team agreed to use PostgreSQL for the first version.", "reason": "Structured project data fits PostgreSQL better.", "participants": ["Ayush", "Rahul", "Priya"], "confidence": 0.91, "needs_confirmation": true }
```

20.2 Evidence Verification Prompt

Input:

- proposed answer
- retrieved Slack evidence
- confirmed memory

Output JSON:

```
{ "support_level": "SUPPORTED", "confidence": 0.88, "contradictions": [], "missing_evidence": [],  
  "safe_to_answer": true }
```

Support levels:

- SUPPORTED
- PARTIALLY_SUPPORTED
- CONFLICTING
- INSUFFICIENT_EVIDENCE

20.3 Conflict Detection Prompt

Input:

- new message
- confirmed memory
- latest Slack context

Output JSON:

```
{ "is_conflict": true, "conflict_type": "technology_choice", "severity": "medium", "explanation": "New  
message mentions MongoDB, but confirmed memory says PostgreSQL.", "recommended_action":  
  "remind_decision" }
```

21. Slack Commands and Interactions

21.1 App Mentions

@AlignOS what did we decide about database?

@AlignOS show project memory

@AlignOS summarize today

@AlignOS show conflicts

@AlignOS reopen database decision

21.2 Optional Slash Commands

/alignos memory

/alignos decisions

/alignos conflicts

/alignos summarize

/alignos help

21.3 Buttons

Decision:

- Confirm
- Edit
- Reject

Conflict:

- Remind Decision
- Reopen Decision
- Ignore

Memory:

- View Details
- Export
- Update
- Archive

22. API/Endpoint Requirements

Backend endpoints:

POST /slack/events

Handles app mentions, message events, URL verification.

POST /slack/interactions

Handles buttons and modals.

POST /slack/commands

Handles optional slash commands.

POST /agent/ask

Internal endpoint for question-answering.

POST /agent/detect-decision

Internal endpoint for decision detection.

POST /agent/detect-conflict

Internal endpoint for conflict detection.

GET /health

Health check.

23. MCP Server Tool Requirements

MCP server should expose:

1. detect_decision
2. save_decision
3. search_memory
4. detect_conflict
5. verify_evidence
6. generate_project_summary
7. reopen_decision
8. log_conflict_action

The backend should act as an MCP client.

The MCP server should not directly handle Slack events. Slack events belong in the Slack backend.

24. Permissions and Scopes

Likely Slack capabilities needed:

- Read app mentions
- Read messages in channels where installed
- Post messages
- Use interactivity
- Use AI assistant/app features
- Use Real-Time Search API if available
- Optional slash commands

Exact scopes will depend on the Slack app configuration and hackathon workspace access.

25. Success Metrics

25.1 Product Metrics

- Number of decisions detected
- Decision confirmation rate
- Number of conflicts detected
- Number of useful answers generated
- Number of “insufficient evidence” refusals
- Reduction in repeated questions
- User clicks on confirmation/action buttons

25.2 Quality Metrics

- Decision detection precision
- Conflict detection precision
- Hallucination rate
- Evidence support score
- User feedback score

25.3 Demo Metrics

For hackathon demo, show:

- At least 1 detected decision
- At least 1 confirmed memory item
- At least 1 evidence-backed answer
- At least 1 detected conflict
- At least 1 no-evidence refusal or uncertainty response

26. Risks and Mitigations

26.1 Risk: Too many false conflict alerts

Mitigation:

- Only alert when confidence is medium/high.
- Use recent context before alerting.
- Add Ignore button.
- Suppress repeated alerts for same topic.

26.2 Risk: AI hallucination

Mitigation:

- RAG evidence checker.
- No-evidence refusal.
- Structured verification output.
- Use confirmed memory as source of truth.

26.3 Risk: Slack permission issues

Mitigation:

- Build fallback using channel history if RTS is unavailable.
- Limit MVP to channels where bot is installed.
- Clearly show workspace installation requirements.

26.4 Risk: Overbuilding MCP

Mitigation:

- Keep Slack event handling in backend.
- Use MCP only for actual tools.
- Start with five core tools.

26.5 Risk: Privacy concerns

Mitigation:

- Store evidence references, not full message history.
- Respect channel access.
- Allow channel-level opt-out.
- Keep admin controls.

27. MVP Build Plan

Phase 1: Slack Bot Foundation

Deliverables:

- Slack app created
- Bot installed in workspace
- Event endpoint working
- App mention reply working
- Basic help command working

Success:

User can type: @AlignOS hello

Bot replies.

Phase 2: Memory Database

Deliverables:

- PostgreSQL/Supabase setup
- decisions table
- evidence_links table
- memory search function
- save decision function

Success:

Backend can save and retrieve a test decision.

Phase 3: MCP Server

Deliverables:

- MCP server running
- MCP tools implemented:
 - detect_decision
 - save_decision
 - search_memory
 - detect_conflict
 - verify_evidence

Success:

Backend can call MCP tools and receive structured JSON.

Phase 4: Real-Time Search RAG

Deliverables:

- Real-Time Search API integration
- Query generator
- Evidence formatter
- Evidence verification prompt

Success:

User asks: @AlignOS what did we decide about database?

Bot searches Slack and answers with evidence.

Phase 5: Decision Confirmation Cards

Deliverables:

- Decision detector on message events
- Slack Block Kit confirmation card
- Confirm/Reject buttons
- Save confirmed decision to DB

Success:

Message: Let's finalize PostgreSQL for v1.

Bot posts: Possible decision detected.

User confirms.

Decision saved.

Phase 6: Conflict Detection

Deliverables:

- Message topic extraction
- Memory comparison
- RTS latest-context check
- Conflict alert card
- Ignore/Reopen/Remind buttons

Success:

After confirming PostgreSQL, a user says: I'll start MongoDB.

Bot detects conflict.

Phase 7: Demo Polish

Deliverables:

- Demo workspace
- Demo channel script
- Clean bot messages
- Error handling
- Short pitch
- Submission screenshots/video

Success:

A judge can understand the product in under 2 minutes.

28. Demo Script

Scene 1: Messy Team Discussion

Team discusses database choice.

Messages:

Ayush: Should we use PostgreSQL or MongoDB?

Priya: PostgreSQL is better because our task data is structured.

Rahul: MongoDB may be faster but PostgreSQL sounds safer.

Ayush: Okay final, PostgreSQL for v1.

Rahul: Agreed.

Scene 2: Decision Detection

AlignOS posts:

Possible decision detected: Use PostgreSQL for v1.

Reason: Structured task data.

[Confirm] [Reject]

Ayush clicks Confirm.

Scene 3: Memory Q&A

User asks:

@AlignOS why did we choose PostgreSQL?

AlignOS replies with evidence-backed answer.

Scene 4: Conflict

Rahul posts:

I'll start MongoDB setup.

AlignOS replies:

Possible conflict detected. Confirmed memory says PostgreSQL for v1.

[Remind Decision] [Reopen Decision] [Ignore]

Scene 5: Project Memory

User asks:

@AlignOS show project memory

AlignOS shows:

- Confirmed decisions
- Open tasks
- Blockers
- Conflicts
- Unresolved questions

29. Judging Positioning

AlignOS should be presented as:

A Slack-native agentic memory layer, not a chatbot.

Key strengths:

1. Uses Slack as the natural work interface.
2. Uses Real-Time Search API for live context.
3. Uses MCP for modular tool execution.
4. Uses RAG verification to reduce hallucination.
5. Builds structured project memory from unstructured conversation.
6. Detects conflicts before they become project failures.

30. Final MVP Definition

The MVP is complete when:

1. User can ask AlignOS questions in Slack.
2. AlignOS can search live Slack context.
3. AlignOS can detect a decision.
4. User can confirm the decision.
5. Confirmed decision is saved in memory.

6. AlignOS can answer future questions from memory + evidence.
7. AlignOS can detect a contradiction against confirmed memory.
8. AlignOS can show a project memory summary.

31. Product Tagline Options

Option 1: Your Slack agent that catches decisions before they disappear.

Option 2: Turn Slack chaos into verified team memory.

Option 3: The live memory and alignment layer for Slack teams.

Option 4: An AI agent that detects decisions, verifies answers, and prevents team misalignment.

Recommended tagline: Turn Slack chaos into verified team memory.

32. Recommended Product Name

Primary recommendation: AlignOS

Alternatives:

- ContextOS
- DecisionGraph
- TeamMemory AI
- SlackSense
- Consensus AI
- ThreadBrain

Recommended final name: AlignOS

Reason: The product's real purpose is not just memory. It is team alignment.